TUGAS TERBIMBING
PENGENALAN LIBRARY NETWORKX

Nama : Nuevalen Refitra Alswando
Kelas : IF-04-01
NIM : 103072430008

Soal 1

soal1.py 2 × networkx.ipynb soal2.py 2

modul03 > terbimbing > soal1.py > ...

You, 5 seconds ago | 1 author (You)

```python
import networkx as nx
import matplotlib.pyplot as plt

G = nx.Graph()

edges = [
    ('Arad', 'Zerind', 75),
    ('Arad', 'Sibiu', 140),
    ('Arad', 'Timisoara', 118),
    ('Zerind', 'Oradea', 71),
    ('Oradea', 'Sibiu', 151),
    ('Timisoara', 'Lugoj', 111),
    ('Lugoj', 'Mehadia', 70),
    ('Mehadia', 'Dobreta', 75),
    ('Dobreta', 'Craiova', 120),
    ('Craiova', 'Rimnicu Vilcea', 146),
    ('Craiova', 'Pitesti', 138),
    ('Rimnicu Vilcea', 'Sibiu', 80),
    ('Rimnicu Vilcea', 'Pitesti', 97),
    ('Sibiu', 'Fagaras', 99),
    ('Fagaras', 'Bucharest', 211),
    ('Pitesti', 'Bucharest', 101),
    ('Giurgiu', 'Bucharest', 90),
    ('Urziceni', 'Bucharest', 85),
    ('Urziceni', 'Hirsova', 98),
    ('Urziceni', 'Vaslui', 142),
    ('Hirsova', 'Eforie', 86),
    ('Vaslui', 'Iasi', 92),
    ('Iasi', 'Neamt', 87)
]

G.add_weighted_edges_from(edges)

print("=== GRAF ROMANIA ===")
print(f"Jumlah node (kota): {G.number_of_nodes()}")
print(f"Jumlah edge (jalur): {G.number_of_edges()}")
print(f"Kota-kota: {list(G.nodes())}")
print()

print("-" * 80)
print("=== SEMUA PASANGAN SHORTEST PATH (ALL PAIRS SHORTEST PATH) ===")
print("-" * 80)

all_shortest_paths = dict(nx.all_pairs_dijkstra_path(G, weight='weight'))
all_shortest_path_lengths = dict(nx.all_pairs_dijkstra_path_length(G, weight='weight'))
```

soal1.py 2 × networkx.ipynb soal2.py 2

modul03 > terbimbing > soal1.py > ...

You, now • Uncommitted changes

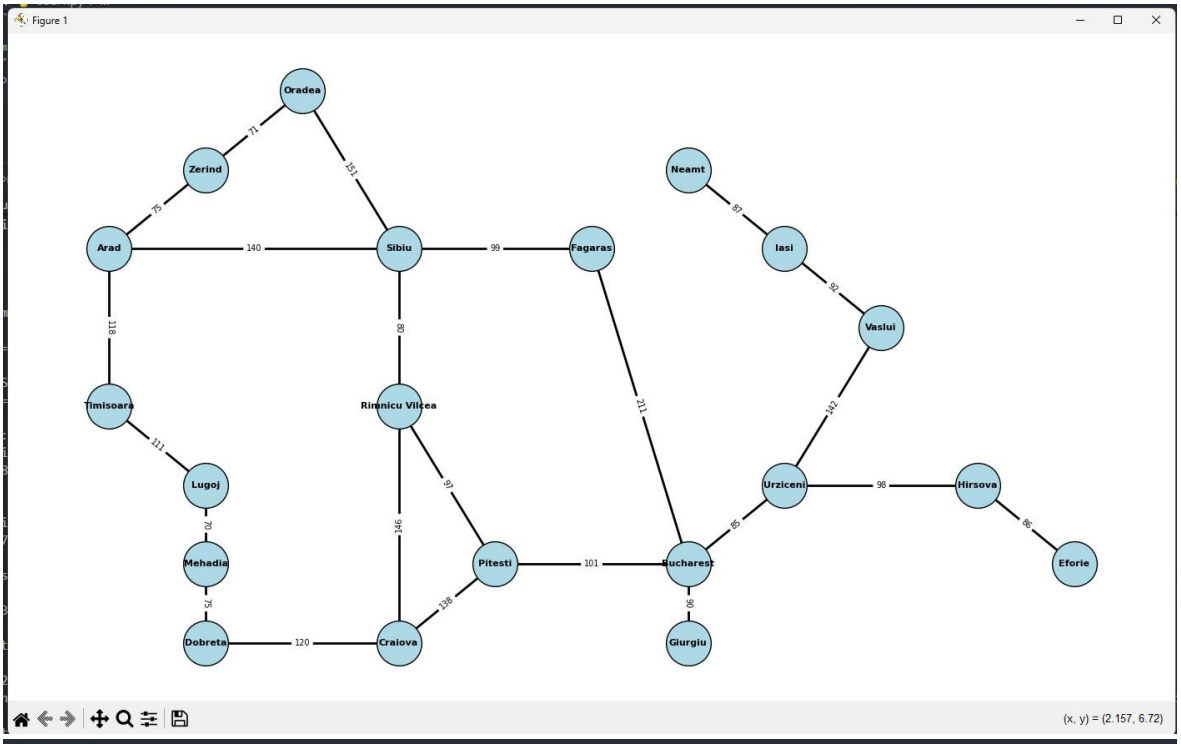
```python
print("\n=== CONTOH SHORTEST PATH PENTING ===")
print("=" * 80)

important_routes = [
    ('Arad', 'Bucharest'),
    ('Arad', 'Iasi'),
    ('Zerind', 'Bucharest'),
    ('Timisoara', 'Neamt')
]

for source, target in important_routes:
    path = nx.dijkstra_path(G, source=source, target=target, weight='weight')
    distance = nx.dijkstra_path_length(G, source=source, target=target, weight='weight')
    path_str = " -> ".join(path)
    print(f"\n{source} ke {target}:")
    print(f"  Rute: {path_str}")
    print(f"  Total jarak: {distance} km")

# Visualisasi graf
pos = {
    "Arad": (0,4),
    "Zerind": (1,5),
    "Oradea": (2,6),
    "Sibiu": (3,4),
    "Timisoara": (0,2),
    "Lugoj": (1,1),
    "Mehadia": (1,0),
    "Dobreta": (1,-1),
    "Craiova": (3,-1),
    "Rimnicu Vilcea": (3,2),
    "Pitesti": (4,0),
    "Fagaras": (5,4),
    "Bucharest": (6,0),
    "Giurgiu": (6,-1),
    "Urziceni": (7,1),
    "Hirsova": (9,1),
    "Eforie": (10,0),
    "Vaslui": (8,3),
    "Iasi": (7,4),
    "Neamt": (6,5)
}

plt.figure(figsize=(14, 8))

nx.draw(
    G,
```

0 4 207 Live Share Reconnect to Discord You, now Ln 48, Col 1 Spaces: 4 UTF-8 CRLF {} Python venv (3.14.3) Go Live Background Prettier

soal1.py 2 | networkx.ipynb | soal2.py 2

modul03 > terbimbing > soal1.py > ...

```python
pos = {
    "Arad": (0,4),
    "Zerind": (1,5),
    "Oradea": (2,6),
    "Sibiu": (3,4),
    "Timisoara": (0,2),
    "Lugoj": (1,1),
    "Mehadia": (1,0),
    "Dobreta": (1,-1),
    "Craiova": (3,-1),
    "Rimnicu Vilcea": (3,2),
    "Pitesti": (4,0),
    "Fagaras": (5,4),
    "Bucharest": (6,0),
    "Giurgiu": (6,-1),
    "Urziceni": (7,1),
    "Hirsova": (9,1),
    "Eforie": (10,0),
    "Vaslui": (8,3),
    "Iasi": (7,4),
    "Neamt": (6,5)
}

plt.figure(figsize=(14, 8))

nx.draw(
    G,
    pos,
    with_labels=True,
    node_size=1500,
    node_color="lightblue",
    edgecolors="black",
    font_size=8,
    font_weight="bold",
    width=2
)

edge_labels = nx.get_edge_attributes(G, 'weight')
nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_size=7)

plt.title("Romania Map Graph - Weighted Graph", fontsize=14, fontweight='bold', pad=20)
plt.axis("off")
plt.tight_layout()
plt.show()
```




```
Arad ke Bucharest:
  Rute: Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
  Total jarak: 418 km

Arad ke Iasi:
  Rute: Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest -> Urziceni -> Vaslui -> Iasi
  Total jarak: 737 km

Zerind ke Bucharest:
  Rute: Zerind -> Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
  Total jarak: 493 km

Timisoara ke Neamt:
  Rute: Timisoara -> Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest -> Urziceni -> Vaslui -> Iasi -> Neamt
  Total jarak: 942 km
```

Kode pada Soal 1 bertujuan untuk memodelkan jaringan kota-kota di negara Romania menggunakan struktur data graf berbobot (*weighted graph*). Program dimulai dengan mengimpor pustaka networkx untuk manipulasi graf dan matplotlib untuk visualisasi. Selanjutnya, dibuatlah sebuah objek graf kosong kemudian diisi dengan daftar sisi (*edges*) yang menghubungkan antar kota beserta bobot jaraknya dalam kilometer, seperti rute dari Arad ke Zerind sejauh 75 km. Setelah graf terbentuk, program menghitung dan menampilkan informasi dasar seperti jumlah kota dan jalur yang tersedia. Kode ini juga mengimplementasikan algoritma Dijkstra untuk mencari jalur terpendek (*shortest path*) antar pasangan kota penting, misalnya dari Arad ke Bucharest. Terakhir, program memvisualisasikan graf tersebut ke dalam bentuk peta jaringan dengan posisi node yang diatur secara manual agar menyerupai letak geografis aslinya, lengkap dengan label jarak pada setiap penghubung kota.

Soal 2

modul03 > terbimbing > soal2.py > ...

You, 5 minutes ago | 1 author (You)

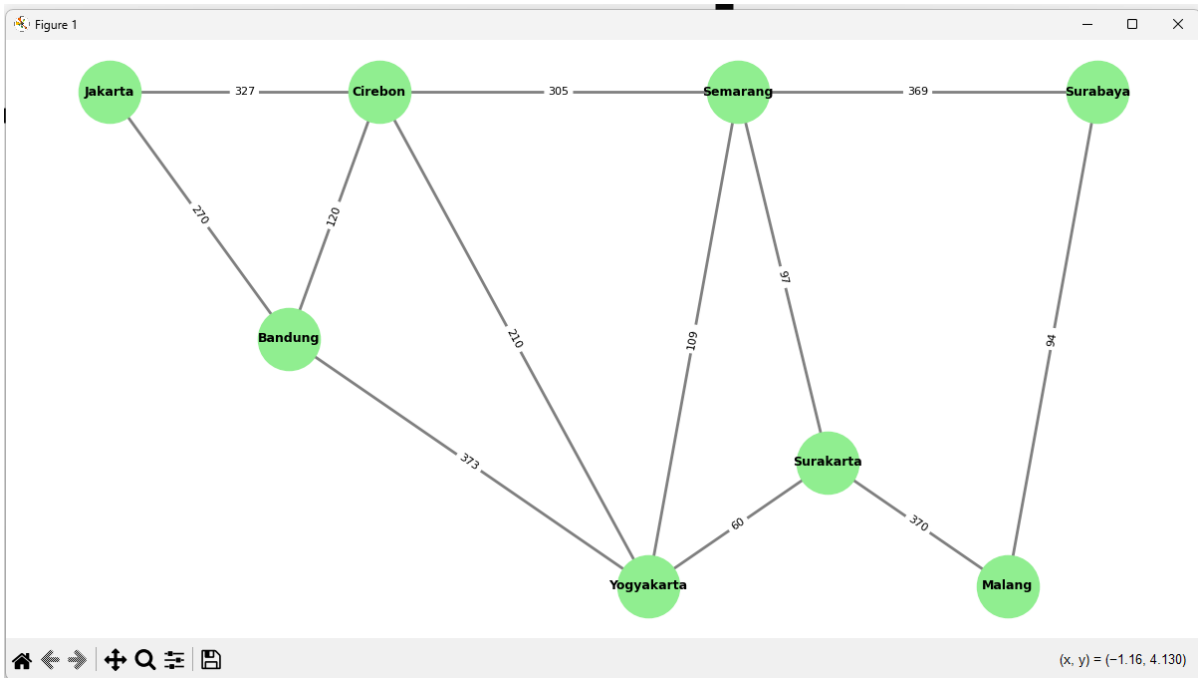
```python
import networkx as nx
import matplotlib.pyplot as plt

G_jawa = nx.Graph()

edges_jawa = [
    ('Jakarta', 'Bandung', 270),
    ('Jakarta', 'Cirebon', 327),
    ('Bandung', 'Cirebon', 120),
    ('Bandung', 'Yogyakarta', 373),
    ('Cirebon', 'Yogyakarta', 210),
    ('Cirebon', 'Semarang', 305),
    ('Yogyakarta', 'Semarang', 109),
    ('Yogyakarta', 'Surakarta', 60),
    ('Semarang', 'Surakarta', 97),
    ('Semarang', 'Surabaya', 369),
    ('Surakarta', 'Malang', 370),
    ('Surabaya', 'Malang', 94)
]

G_jawa.add_weighted_edges_from(edges_jawa)

print("=== GRAF KOTA-KOTA JAWA ===")
print(f"Jumlah node: {G_jawa.number_of_nodes()}")
print(f"Jumlah edge: {G_jawa.number_of_edges()}")
print()

print("=== SHORTEST PATH ===")
print(f"Jakarta ke Surabaya: {nx.shortest_path(G_jawa, 'Jakarta', 'Surabaya', weight='weight')}")
print(f"Jarak: {nx.shortest_path_length(G_jawa, 'Jakarta', 'Surabaya', weight='weight')} km")
print()

print(f"Bandung ke Malang: {nx.shortest_path(G_jawa, 'Bandung', 'Malang', weight='weight')}")
print(f"Jarak: {nx.shortest_path_length(G_jawa, 'Bandung', 'Malang', weight='weight')} km")
print()

print(f"Jakarta ke Yogyakarta: {nx.shortest_path(G_jawa, 'Jakarta', 'Yogyakarta', weight='weight')}")
print(f"Jarak: {nx.shortest_path_length(G_jawa, 'Jakarta', 'Yogyakarta', weight='weight')} km")

pos = {
    'Jakarta': (0, 4),
    'Bandung': (2, 2),
    'Cirebon': (3, 4),
    'Yogyakarta': (6, 0),
    'Surakarta': (8, 1),
    'Semarang': (7, 4)
```

soal1.py 2 | Readme.md | networkx.ipynb | soal2.py 2

modul03 > terbimbing > soal2.py > ...

```python
    ('Semarang', 'Surabaya', 369),
    ('Surakarta', 'Malang', 370),
    ('Surabaya', 'Malang', 94)
]

G_jawa.add_weighted_edges_from(edges_jawa)

print("=== GRAF KOTA-KOTA JAWA ===")
print(f"Jumlah node: {G_jawa.number_of_nodes()}")
print(f"Jumlah edge: {G_jawa.number_of_edges()}")
print()

print("=== SHORTEST PATH ===")
print(f"Jakarta ke Surabaya: {nx.shortest_path(G_jawa, 'Jakarta', 'Surabaya', weight='weight')}")
print(f"Jarak: {nx.shortest_path_length(G_jawa, 'Jakarta', 'Surabaya', weight='weight')} km")
print()

print(f"Bandung ke Malang: {nx.shortest_path(G_jawa, 'Bandung', 'Malang', weight='weight')}")
print(f"Jarak: {nx.shortest_path_length(G_jawa, 'Bandung', 'Malang', weight='weight')} km")
print()

print(f"Jakarta ke Yogyakarta: {nx.shortest_path(G_jawa, 'Jakarta', 'Yogyakarta', weight='weight')}")
print(f"Jarak: {nx.shortest_path_length(G_jawa, 'Jakarta', 'Yogyakarta', weight='weight')} km")

pos = {
    'Jakarta': (0, 4),
    'Bandung': (2, 2),
    'Cirebon': (3, 4),
    'Yogyakarta': (6, 0),
    'Surakarta': (8, 1),
    'Semarang': (7, 4),
    'Surabaya': (11, 4),
    'Malang': (10, 0)
}

plt.figure(figsize=(12, 6))
nx.draw(G_jawa, pos, with_labels=True, node_color='lightgreen',
        node_size=2000, font_size=9, font_weight='bold',
        edge_color='gray', width=2)
edge_labels = nx.get_edge_attributes(G_jawa, 'weight')
nx.draw_networkx_edge_labels(G_jawa, pos, edge_labels=edge_labels, font_size=8)
plt.title('Graf Kota-Kota Besar di Pulau Jawa')
plt.axis('off')
plt.tight_layout()
plt.show()
```

You, last week • update …



Kode pada Soal 2 memiliki konsep yang serupa namun diterapkan pada konteks lokal, yaitu memodelkan jaringan transportasi antar kota besar di Pulau Jawa, Indonesia. Program mendefinisikan graf baru yang menghubungkan delapan kota utama seperti Jakarta, Bandung, Surabaya, dan lainnya dengan bobot jarak yang diambil dari studi kasus peta yang diberikan. Setelah struktur graf dibangun, fokus utama kode ini adalah melakukan analisis rute tercepat menggunakan fungsi shortest_path dari library NetworkX. Program secara spesifik menghitung dan mencetak rute terpendek serta total jaraknya untuk tiga perjalanan lintas kota, yaitu dari Jakarta ke Surabaya, Bandung ke Malang, dan Jakarta ke Yogyakarta. Sebagai hasil akhir, kode ini menampilkan visualisasi graf yang disusun posisinya (pos) agar merepresentasikan peta pulau Jawa secara kasar, memudahkan pengguna untuk melihat hubungan konektivitas dan jarak antar kota secara visual.